



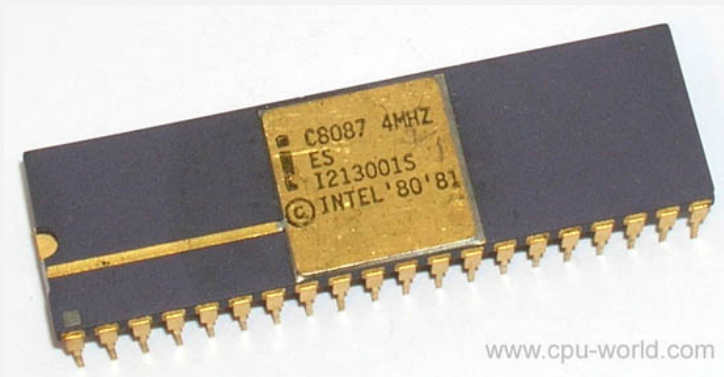
Числа с плавающей запятой





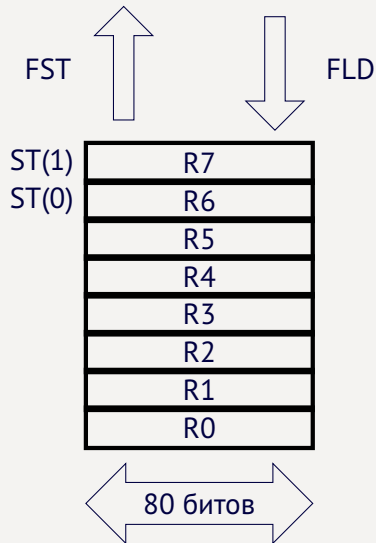
x87 FPU (1980)

- Первоначально отдельная микросхема Intel-8087, которая расширяла возможности процессора Intel-8086, так называемый “сопроцессор”.
- Производительность около 50000 Флопс (Floating Point Operations Per Second)





x87 FPU



инициализация сопроцессора

...

<--- начало работы сопроцессора

FLD addr1

FLD addr2

FADD ST(0), ST(1)

FSTP addr

<--- конец работы сопроцессора

...

проверка состояния сопроцессора



Векторные регистры

- MMX (1997 г.) — регистры MM0-MM7 по 64 бита. Общие с регистрами FPU
- SSE — регистры XMM0-XMM7 по 128 битов. Специальный набор команд для работы с числами с плавающей запятой
 - SSE1(1999 г.) — только для single precision (**float**)
 - SSE2(2000 г.) — все команды для double precision (**double**)



Представление в памяти

$$(-1)^S \times M \times B^E$$

- S — знак
- M — мантисса
- B — основание
- E — порядок (экспонента)



Нормальная форма

- **Нормальной формой** числа с плавающей запятой называется такая форма, в которой мантисса (без учёта знака) в десятичной системе находится на полуинтервале $[0; 1)$
 - 0,0001 — как построить нормальную форму?



Нормальная форма

- **Нормальной формой** числа с плавающей запятой называется такая форма, в которой мантисса (без учёта знака) в десятичной системе находится на полуинтервале $[0; 1)$
 - 0,0001 — как построить нормальную форму?
 - $0,0001 \times 10^0$?



Нормальная форма

- **Нормальной формой** числа с плавающей запятой называется такая форма, в которой мантисса (без учёта знака) в десятичной системе находится на полуинтервале $[0; 1)$
 - 0,0001 — как построить нормальную форму?
 - $0,0001 \times 10^0$?
 - $0,1 \times 10^{-3}$?



Нормализованная форма

- **Нормализованной формой** числа с плавающей запятой называется такая форма, в которой у числа слева от запятой ровно одна значащая цифра
 - $0,57 = 5,7 \times 10^{-1}$
 - $232,14 = 2,3214 \times 10^2$



Нормализованная форма

- **Нормализованной формой** числа с плавающей запятой называется такая форма, в которой у числа слева от запятой ровно одна значащая цифра
 - $0,57 = 5,7 \times 10^{-1}$
 - $232,14 = 2,3214 \times 10^2$
 - **А ноль?**



Двоичная система счисления

Вспомним, что в компьютере система счисления не десятичная

- $\frac{3}{4} = ?_2$
- $\frac{1}{10} = ?_2$
- $1,01001_2 = ?_{10}$
- $1,1101_2 = ?_{10}$



Двоичная система счисления

Вспомним, что в компьютере система счисления не десятичная

- $\frac{3}{4} = 0,11_2$
- $\frac{1}{10} = ?_2$
- $1,01001_2 = ?_{10}$
- $1,1101_2 = ?_{10}$



Двоичная система счисления

Вспомним, что в компьютере система счисления не десятичная

- $\frac{3}{4} = 0,11_2$
- $\frac{1}{10} = 0,0(0011)_2$
- $1,01001_2 = ?_{10}$
- $1,1101_2 = ?_{10}$



Двоичная система счисления

Вспомним, что в компьютере система счисления не десятичная

- $\frac{3}{4} = 0,11_2$
- $\frac{1}{10} = 0,0(0011)_2$
- $1,01001_2 = 1,28125_{10}$
- $1,1101_2 = ?_{10}$



Двоичная система счисления

Вспомним, что в компьютере система счисления не десятичная

- $\frac{3}{4} = 0,11_2$
- $\frac{1}{10} = 0,0(0011)_2$
- $1,01001_2 = 1,28125_{10}$
- $1,1101_2 = 1,8125_{10}$



Одинарная точность (float)



$$(-1)^S \times M \times B^{E-127}$$

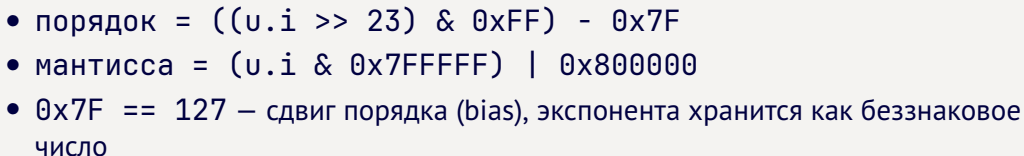


Union в C

- Интерпретация **одного и того же** значения в соответствии с разными **типами**
 - «каламбур типизации»

```
union Some {  
    int i;  
    float f;  
} u;
```

- Реальный use case — http://wiki.libsdl.org/SDL_Event
- В C такое чтение «чужого» поля — *implementation-defined*; в C++ это неопределённое поведение





Точность вычислений

- Мантисса: 23 бита + 1 неявный
 - $\varepsilon \sim \frac{1}{2^{24}} \sim 5,96 * 10^{-8}$
- Точность составляет приблизительно **7 значащих цифр**



Специальные представления

- **Ноль** (нет нормализованного представления)
- **Бесконечности**
- **NaN** (Not a Number)



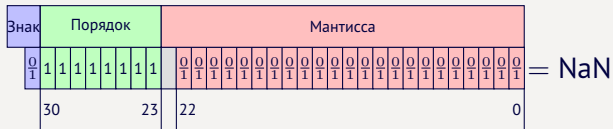
Специальные значения: ноль со знаком



- Представление
 - порядок — все биты 0
 - мантисса — все биты 0
- Операции
 - $(-0) * (-0) = +0$
 - $(-0) + (-0) = -0$



Специальные значения: NaN

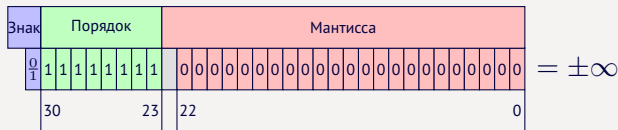


Является результатом арифметических операций, если во время их выполнения произошла ошибка

- Представление
 - порядок — все биты 1
 - мантисса **ненулевая**, иначе это бесконечность
- Операции
 - Результат операции с NaN всегда равен NaN



Специальные значения: бесконечности



- Представление
 - порядок — все биты 1
 - мантисса — все биты 0
- Операции
 - Результат операции с $\pm\infty$ **почти всегда** равен $\pm\infty$



Как получить NaN?

- $\frac{\pm 0}{\pm 0} = \frac{\pm \infty}{\pm \infty} = NaN$
- $0 \times \infty = NaN$
- $\sqrt{x} = NaN, x < 0$
- $+\infty + (-\infty) = NaN$
- Есть и другие способы
- **Результат операции с NaN всегда равен NaN**



Как получить бесконечность?

- Переполнение
- Деление ненулевого числа на ноль
- *Попробуйте найти примеры получения бесконечности при переполнении*



Денормализованные числа



- Пусть $\varepsilon_1, \varepsilon_2$ — очень маленькие нормализованные числа
- Их разность $\text{diff} = \varepsilon_1 - \varepsilon_2$ может оказаться меньше наименьшего нормализованного числа
 - Считать $\text{diff} = 0$ — тогда возможно деление на ноль
 - Считать $\text{diff} \neq 0$ — тогда чему она равна и как её представить?
- Ответ на эти вопросы — **денормализованные числа**



Денормализованные числа

Находятся ближе к нулю, чем любое нормализованное число

- Порядок — все биты 0 (как у нуля)
- $M = 0 \Rightarrow$ трактуется как 0
- $M! = 0 \Rightarrow$ это не 0:
 - Неявный бит мантииссы равен **0** (вместо 1)
 - Эффективная экспонента: $E = E_{\min}$

Особенность: могут обрабатываться аппаратно, но со штрафом производительности (в некоторых режимах, FTZ/DAZ, — сбрасываются в ноль)

CERT Secure Coding: Do not use denormalized numbers



Свойства операций

- $a + b = b + a$
- $a * b = b * a$
- $(a + b) + c = a + (b + c)$
- $(a * b) * c = a * (b * c)$
- $a * (b + c) = (a * b) + (a * c)$



Свойства операций

- $x = x$
 - $\text{NaN} = \text{NaN} \Rightarrow \text{false}$
- $x + 0 = x$
 - $-0 + 0 = 0$
- $x * 0 = 0$
 - $x < 0 \Rightarrow x * 0 = -0$



Уровень 0: сравнение через ==

Из-за ошибок округления числа, которые «должны быть равны», часто отличаются незначительно:

```
double a = 0.15 + 0.15;  
double b = 0.1 + 0.2;  
if (a == b) // может быть false!  
if (a >= b) // тоже может быть false!
```

- Разные корректные способы вычисления дают **разные битовые представления**
- Побитовое сравнение == для чисел с плавающей точкой **не работает**



Уровень 1: абсолютный порог

Сравниваем не сами числа, а **модуль разности** с порогом ϵ :

if (fabs(a - b) < 0.00001f) // *лучше, чем ==, но можно лучше*

- Порог «на глаз» никак не связан с масштабом чисел
- Для **малых** чисел порог слишком велик — «разные» числа признаются равными
- Для **больших** чисел порог меньше ошибки округления — «равные» числа признаются разными

Нужен порог, который **масштабируется** вместе с величиной чисел.



Уровень 2: относительная погрешность

Сравниваем **относительную** ошибку:

if (fabs((a - b) / b) < 0.00001f) *// лучше, но не идеально*

- Порог автоматически масштабируется с величиной чисел
- Результат **не коммутативен**: $\text{near}(a, b) \neq \text{near}(b, a)$



Относительная погрешность: особые случаи

- $a = b = 0$: деление $0/0$ даёт NaN
- $b = 0$: деление даёт ∞ — даже когда a очень мало
- a и b очень малы и по разные стороны от нуля: сравнение даёт false, хотя числа «рядом»

Вывод: для «околонулевых» значений классическая относительная погрешность теряет смысл, нужна отдельная логика.



Устойчивая функция сравнения

```
bool nearlyEqual(float a, float b, float epsilon) {  
    float absA = fabsf(a), absB = fabsf(b);  
    float diff = fabsf(a - b);  
    if (a == b) { // точное совпадение, в т.ч. ±inf  
        return true;  
    } else if (a == 0 || b == 0 || (absA + absB < FLT_MIN)) {  
        // около нуля относительная погрешность бессмысленна  
        return diff < (epsilon * FLT_MIN);  
    } else { // обычная относительная погрешность  
        return diff / fminf(absA + absB, FLT_MAX) < epsilon;  
    }  
}
```



Уровень 3: сравнение как целых чисел

Идея: рассматривать числа как **дискретные шаги** формата

- Порог — максимальное число представимых значений между a и b
- IEEE 754 сохраняет порядок при интерпретации битов как целых **для неотрицательных чисел**; у отрицательных порядок обратный
- Порог **неявно** масштабируется с величиной чисел
- Концептуально просто, но требует преобразования $float \leftrightarrow int$ в языке



Что запомнить

- **`==`** — не работает из-за ошибок округления
- **Абсолютный порог** — шаг вперёд, но не учитывает масштаб
- **Относительная погрешность** — правильнее, но требует аккуратной обработки нулей и денормалей
- **Сравнение как целых** — ясная альтернатива, если нужны «шаги» между соседними значениями



Fast Inverse Square Root

- **Input:** число с плавающей точкой одинарной точности
- **Output:** $1/\sqrt{x}$

https://www.youtube.com/watch?v=p8u_k2LIZyo — Quake III



Fast Inverse Square Root

Алгоритм:

- вычисляет половину значения числа и сохраняет для дальнейшего использования
- трактует 32-битное с плавающей запятой как 32-битное целое:
 - производит логический сдвиг вправо на один бит
 - вычитает число из магической константы `0x5f3759df`
- (в трактовке результата как 32-битного с плавающей запятой на этом этапе получается первое приближение обратного квадратного корня исходного числа)
- вычисляет одну итерацию метода Ньютона

Полезные ссылки



- [What Every Computer Scientist Should Know About Floating-Point Arithmetic](#)
- [Тест про стандарт IEEE 754](#)